Архитектура расчёта квартального плана

Сервис планирования команд · документ для участников проекта · 17 сентября 2026

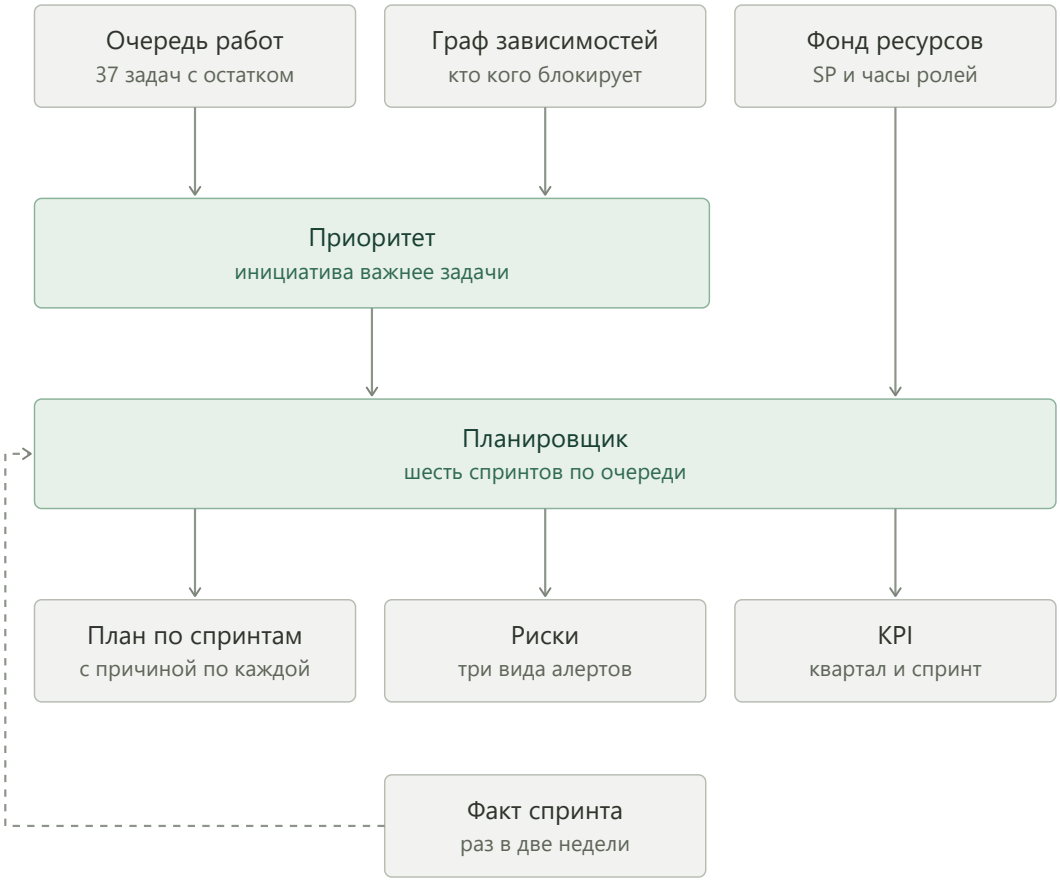
Как устроен расчёт плана: из чего он собирается, какой приём работает на каждом этапе и почему выбран именно он. Звёздная карта и Bus Factor в этот документ не входят — они описываются отдельно.

Идея в двух абзацах

Сервис не раскладывает задачи по спринтам «по объёму». Он ведёт учёт двух независимых ресурсов — очков команды и часов по каждой роли отдельно — и на каждом шаге проверяет оба. Задача, для которой в команде нет нужного специалиста, не встанет в план, даже если свободных очков сколько угодно.

Второе важное решение: единицей ценности считается инициатива PRODF, а не отдельная задача. Главный показатель квартала измеряет завершённые инициативы целиком, поэтому сделать 80 % работ по каждой из пятнадцати инициатив — это ноль по метрике, а закрыть пять инициатив полностью — треть. Приоритет считается на уровне инициативы, задачи внутри неё выстраиваются по зависимостям.

Схема расчёта



Узлы — этапы расчёта, стрелки — что передаётся дальше. Зелёным выделены два узла, где принимаются решения.

Как читать схему

Верхний ряд — три подготовительных блока. Они собираются из одного и того же Excel-листа, но отвечают на разные вопросы: **что** делать, **в каком порядке** и **чем**. Каждый считается независимо, поэтому их можно разрабатывать параллельно и тестировать по отдельности.

Средние два узла — ядро. Приоритет отвечает на вопрос «чем жертвуем первым», планировщик — «что в каком спринте и за чей счёт». Обратите внимание, что фонд ресурсов идёт в планировщик напрямую, минуя приоритет: важность работы не зависит от того, хватает ли на неё людей. Сначала решаем, что важно, потом смотрим, что поместилось. Иначе получится план, который оправдывает дефицит вместо того, чтобы его показывать.

Нижний ряд — три выхода, и все три обязательны по условиям задачи: сам план с причинами по каждой задаче, предупреждения о рисках и показатели выполнения.

Пунктирная стрелка слева — цикл пересчёта. Раз в две недели приходит факт спринта, он обновляет остатки работ и запускает планировщик заново, но только на оставшихся спринтах. Прошедшие спринты замораживаются, а база сравнения, зафиксированная в самом начале, не меняется никогда — иначе показатели можно подогнать задним числом.

Что важно донести до всех участников: узлы связаны только через данные, а не через общее состояние. Каждый получает на вход готовый объект и отдаёт готовый объект. Поэтому любой узел можно переписать, не трогая соседние, и проверить отдельным тестом.

Что показали данные

Прежде чем выбирать алгоритм, мы посчитали баланс по 37 активным задачам (31 в статусе ToDo, 6 в работе).

Показатель	Требуется	Доступно за квартал
Story Points	240	329
Человеко-часы	5 983	14 400

По общим цифрам всё помещается. Но если специалистов брать строго из своей команды, **2 590 часов из 5 983 выполнять некому** — это 43 % работы. Дефицит делится на две принципиально разные части:

Вид дефицита	Часов	Что с этим делать
Роль есть в компании, но в другой команде	1 939	Заимствовать специалиста, пометив это в объяснении и в рисках
Роли нет ни в одной команде	651	Закрыть невозможно: показываем как риск и основание для переноса

Внутри второй группы 455 часов приходятся на роль «Руководитель проекта», остальное — 1С-роли и поддержка. Это и есть «намеренно заложенный дефицит», о котором предупреждает онбординг. Инструмент должен его назвать поимённо, а не молча выдать усечённый список задач.

Связи между задачами почти ничего не решают

Показатель	Значение
Строк в таблице зависимостей	16
Корректных уникальных связей	12
Активных задач, участвующих хотя бы в одной связи	11
Активных задач вне графа полностью	26
Самая длинная цепочка	4 задачи
Связей между разными командами	4

Отсюда прямое следствие для выбора алгоритма: одной топологической сортировки недостаточно. Она упорядочивает 11 задач из 37, а для остальных 26 любой порядок одинаково допустим. Самая длинная цепочка — 4 задачи при шести спринтах, то есть связи почти никогда не являются тем, что упирается. Упираются часы конкретных ролей, и именно их учёт составляет содержательную часть решения.

Этап и приём: общая таблица

Этап	Что применяется	Состояние
Разбор Excel	Поиск блоков по тексту заголовка, разворот матрицы сметы в длинную таблицу	готово
Нормализация ролей	Словарь замен из конфигурации	готово
Валидация	Операции над множествами и группировки	готово
Фонды ресурсов	Группировка с суммированием по спринтам, командам и ролям	частично
Граф зависимостей	Направленный граф, поиск циклов обходом в глубину, топологическая сортировка алгоритмом Кана, длина хвоста динамикой по обратному порядку	нет
Приоритет	Взвешенная сумма нормированных признаков, сортировка по составному ключу	нет
Планировщик	Жадная укладка по спринтам с проверкой ресурсов и нижней оценкой длительности	нет
Объяснения	Журнал событий, без вычислений	нет
Риски	Сравнение с базой, обход потомков в графе, сопоставление спроса и фонда	нет
Показатели	Арифметика по зафиксированной базе	нет

Единственное место, где нужна внешняя библиотека, — граф зависимостей. Всё остальное это группировки в pandas и обычные циклы по словарям: объём маленький, полный расчёт занимает доли секунды.

Планировщик

Почему жадная укладка, а не решатель

Формально это календарное планирование с ограниченными ресурсами — переборная задача, точное решение ищут математическими решателями. Мы сознательно их не берём. Во-первых, за понятность причин включения и переноса задач даётся 15 баллов, а решатель выдаёт матрицу чисел без единого

«почему». Во-вторых, жюри должно уметь проверить план руками. Жадный проход по приоритету объясняет сам себя: на каждом шаге известно, какая задача рассматривается и какой именно ресурс у неё кончился.

Что и как списывается

Часы — по каждой роли отдельно. Задача это не «324 часа», а набор требований: 130 часов .NET, 130 Java, 32 архитектора, 32 девопса. Перерасход одной роли не уменьшает работу другой. Фонд живёт в разрезе спринт × команда × роль.

Story Points — по команде, списываются в спринте, когда задача закрылась целиком: метрика точности спринта считает реально сделанные очки, а сделанными они становятся при статусе Done.

> **Лимит очков проверяется накопительно.** Ёмкость Team-Platform — 7.2 очка на спринт, а в её бэклоге лежат задачи на 8 и 11 очков. При жёсткой проверке «сумма очков в спринте не больше 7.2» такие задачи невыполнимы в принципе, и алгоритм будет вечно их отодвигать. Поэтому проверяется сумма очков, закрытых к концу спринта N, против ёмкости за N спринтов. Крупная задача берёт долг у следующего спринта и возвращает его, за квартал баланс сходится.

Состояние расчёта

```
hours_left[sprint][team][role]   свободные часы роли
sp_left[sprint][team]            свободные очки (накопительно)
remaining[task][role]            сколько работы осталось
finish[task]                     спринт завершения (0 = уже Done)
```

Плюс журнал — список записей «задача, спринт, роль, часы, чья команда дала часы». Из журнала потом собирается и план, и объяснения, и график загрузки ролей.

Основной цикл

```
для s = 1..6:
    очередь = задачи, отсортированные по приоритету

    для task в очередь:
        если remaining[task] пусто: пропустить

        # 1. Ворота зависимостей
        если предшественник имеет finish > s-1 или ещё не запланирован:
            записать причину "ждёт задачу X"; пропустить

        # 2. Проверка выполнимости до конца квартала
        need = max по ролям ceil(remaining[task][role] / фонд_роли_за_спринт)
        если s + need - 1 > 6:
            статус = "перенос"; причина = роль-виновник; пропустить
            # ресурс НЕ тратим

        # 3. Списание часов
        для role, hours в remaining[task]:
            take = min(hours, hours_left[s][team][role])
            если take < hours:
                take += заимствование той же роли у других команд
            remaining[task][role] -= take
            записать в журнал

        # 4. Завершение
        если remaining[task] обнулился:
            если sp_left[s][team] >= SP задачи (накопительно):
                finish[task] = s; списать очки
            иначе:
                finish[task] = ближайший спринт, где очки есть
```

Четыре правила, на которых всё держится

Ворота зависимостей. Онбординг требует зазор: блокируемая задача не может получить ни одного часа в том же спринте, где завершается блокирующая. Условие «предшественник завершён не позже предыдущего спринта» даёт этот зазор автоматически. Задачи со статусом Done получают нулевой спринт завершения и никого не держат.

Проверка выполнимости — самый важный шаг. Без неё жадный алгоритм размажет ресурс: начнёт десяток задач, не закончит ни одной, и по метрике инициатив получится ноль. Поэтому перед списанием часов считается нижняя оценка длительности: по каждой роли остаток делится на фонд за спринт с округлением вверх, берётся максимум. Если конец выпадает за шестой спринт, задача не трогается вовсе, а её часы уходят следующей по приоритету, которая успевает.

Заимствование. Включается, когда в своей команде роли нет или не хватает часов. Порядок обхода делает его безопасным: к моменту, когда задача просит часы у чужой команды, все более приоритетные задачи этой команды в этом спринте уже своё взяли, и отдаётся только остаток. Чья команда дала часы, фиксируется в журнале — это отдельная строка объяснения и отдельное предупреждение. Без заимствования план не построится вообще: 1 939 часов приходятся на роли, которых в своей команде нет.

Детерминированность. При равенстве приоритетов порядок решается идентификатором задачи, чтобы два запуска давали идентичный план. Жюри запускает сервис не один раз.

Очередь приоритета

Сортировка трёхуровневая, каждый уровень решает свою задачу.

Уровень	Ключ	Зачем
1	Вес инициативы PRODF	Показатель квартала считает инициативы целиком, поэтому они входят в план пакетом
2	Топологический уровень внутри инициативы	Предшественники всегда впереди
3	rung задачи, затем task_id	Воспроизводимость плана между запусками

Вес инициативы складывается из четырёх слагаемых: оценка заказчика `rung` (берём максимум среди задач инициативы — внутри он не всегда одинаков, у PRODF-7126 задачи идут с 88 и 92), доля уже вложенной работы (брошенная задача в статусе InProgress — это потерянные деньги), длина хвоста зависимостей и штраф за общий объём в часах: при равной ценности дешёвая инициатива идёт вперёд.

Веса выносятся в `config/assumptions.yaml`. На демонстрации это сильный ход: меняем вес ползунком и показываем, как перестраивается план и какие инициативы спасаются вместо других.

Что получает каждая задача на выходе

Статус	Что означает и что показываем
Запланирована	Спринты с A по B, разбивка часов по ролям и спринтам, пометка о заимствованных часах
Перенесена	Не помещается до конца квартала. Причина называет роль и арифметику: «нужно 272 часа Java, своих 0, у доноров свободно 140 — не успевает»
Заблокирована	Сама поместилась бы, но предшественник перенесён. Причина ссылается на конкретную задачу
Невозможно укомплектовать	Требуется роль, которой нет ни в одной команде. Таких часов 651

Проверка на реальных данных

Team-Platform, семь активных задач, ёмкость 7.2 очка на спринт и такой фонд часов:

Роль	Часов на спринт	Роль	Часов на спринт
Архитектор решений	80	Разработчик Android	80
Разработчик .net	80	Разработчик IOS	80
Тестировщик	80	Девопс	40

Java-разработчика, аналитика, автотестировщика и руководителя проекта в команде нет вообще.

Самая приоритетная задача — DB-202, оценка заказчика 90, уже в работе. Её предшественник DB-201 закрыт, ворота открыты. Остаток после списания факта: 90 часов .NET, 106 Java, 21 архитектор, 30 девопс.

В первом спринте алгоритм закрывает архитектора и девопса целиком, .NET упирается в потолок 80 часов из 90, а Java берёт заимствованием — доноры есть у Team-L, Team-K, Team-Alpha и Team-Beta. Во втором спринте уходят последние 10 часов .NET, задача закрывается и списывает 8 очков при накопленной ёмкости 14.4 — проходит. Следом открывается DB-203, которую DB-202 держала.

И здесь же вылезает то, ради чего всё строится: семь задач Team-Platform стоят 45 очков против 43.2 доступных, а 530 часов из 1 078 приходятся на четыре роли, которых у команды нет. План обязан что-то отодвинуть и назвать, что именно и почему.

> **Отдельная находка.** Задача MP-102 числится в работе, хотя её блокирует ещё не завершённая DB-202. Работу начали раньше, чем разрешают связи. Планировщик такое не чинит, но обязан подсветить — это готовый пример предупреждения о каскадном сдвиге.

Риски и показатели

Предупреждение	Как вычисляется
Красный: срыв инициативы	Прогнозный спринт завершения последней задачи инициативы больше шестого
Жёлтый: каскадный сдвиг	Разница между базой и текущим планом по спринтам, затем обход потомков в графе — показать, кого потянуло
Оранжевый: дефицит роли	Спрос по роли на следующий спринт против фонда часов этой роли

Показатель	Расчёт
Предсказуемость квартала	Инициативы, завершённые за 12 недель, к инициативам в первоначальном плане
Точность спринта	Фактически сделанные очки к изначально запланированным на этот спринт

Оба показателя считаются относительно базы — снимка, сделанного при первом расчёте. База не пересчитывается никогда. Прогнозные значения и фактические результаты хранятся в разных полях и в интерфейсе подписываются по-разному.

Допущения

Все они выносятся в `config/assumptions.yaml` и описываются в README — этого требуют условия задачи.

> **Направление связей проверено по данным.** Все три типа (`has to be done before`, `is required for`, `depends on`) читаются одинаково: первая задача блокирует вторую. Доказательство лежит в самом датасете: пара SRV-4015 → SRV-4016 записана дважды, один раз как `depends on`, второй как `is required for`. Если бы эти типы читались в разные стороны, две задачи заблокировали бы друг друга — замкнутый круг, которого в корректных данных быть не может.

Допущение	Обоснование
Больший rung означает более высокий приоритет	Прямого указания нет; вынесено флагом в конфигурацию
Между связанными задачами зазор в один спринт	Прямое требование онбординга
80 часов на полную ставку уже включают коэффициент 0.8	Прямое указание онбординга: второй раз коэффициент не применяем
Очки задачи в работе считаются пропорционально остатку часов	Иначе наполовину сделанная задача приносит полные очки и портит метрику
Задача может растягиваться на несколько спринтов	Прямой пример из онбординга
Заимствование специалистов между командами разрешено	Без него 43 % часов неисполнимы и плана не существует
Соответствие проверяется по роли, стек — вторично	У задач нет поля с требуемыми технологиями

Модули и порядок работ

К тому, что уже написано (`importer`, `normalizer`, `validator`, `capacity`, `dataset`, `assumptions`), добавляется шесть файлов:

Файл	Ответственность
dependencies.py	Граф связей, циклы, топологический порядок, длина хвоста
scoring.py	Вес инициатив и очередь задач
scheduler.py	Укладка по спринтам и причины решений
risks.py	Три вида предупреждений
kpi.py	Показатели выполнения
baseline.py	Фиксация базы и приём факта спринта

Поток объектов: `Dataset` → `PlanningInput` (очередь, граф, фонды, веса) → `QuarterPlan` (строки плана с причинами и непоместившиеся работы) → `Baseline` и `SprintFact` → риски и показатели. Каждый переход — отдельная функция с одним входом и одним выходом, поэтому любой узел проверяется тестом изолированно.

Порядок работ совпадает с порядком узлов на схеме: граф зависимостей, фонды по спринтам, приоритет, планировщик, объяснения, база и пересчёт, риски и показатели, интерфейс. Первые четыре шага дают проверяемый план одной команды — после них уже есть что показывать.

Документ описывает планируемую реализацию. Цифры получены на датасете «Датасет(обновленный)»: 45 задач, 37 активных, 6 команд, 30 инженеров, 12 корректных связей.